

DOOBIAN CORE

Project Doctrine

Doctrine v1.0 — Established May 2026

*The binding statement of purpose, design, and limits for Doobian Core.
Eight articles. Each clause is intended to be falsifiable — checked against
the project's actual behavior — and to be cited when decisions are hard.*

ARTICLE I

Mission

Works, and improves. For all people.

Three clauses, each load-bearing. The software functions — not in promise, in practice. It gets better over time, because maintenance is the work, not an afterthought. It serves everyone, not a chosen subset. Every other principle in this doctrine flows from these three. When a hard decision has to be made and the answer isn't obvious, the project asks three questions: Does this serve the user? Does this make the software work better, or worse? Does this narrow who it's for? Those questions resolve most ambiguity without further appeal.



ARTICLE II

The User Chooses

No user is locked into one style of operating system over another.

The Linux ecosystem has long forced a choice that does not need to be a choice — systemd or not, Wayland or Xorg, this init or that, this network stack or another. Doobian Core treats every such fork as a configuration decision, not an identity. The user picks. The blessed templates pick for users who do not want to. The Custom Build path serves users who want to choose every component themselves. None of these is the “real” Doobian; they are all Doobian, equally.



ARTICLE III

Modularity

Every system component a user might reasonably want to swap is swappable.

Init system, network manager, display server, session manager, sound server, logger, scheduler, bootloader. If a reasonable person would want to choose, the project provides a way. Modularity is not a feature; it is the architecture. The work of supporting it — keeping the matrix tested, the templates honest, the tooling aware — is the work the project signs up for. A version of Doobian that lost its modularity would no longer be Doobian.



ARTICLE IV

Granularity

Choice goes as deep as the user wants to go.

Modularity at coarse resolution — “pick GNOME or KDE” — is not enough. Doobian operates at the level of the actual building blocks: not “a desktop” but a specific display server, a specific compositor, a specific session manager. Users who want a single decision get blessed templates. Users who want to assemble a system component-by-component get the Custom Build path. The granularity serves power

users, but it also serves the project: it forces the architecture to be honest about its dependencies, and it makes the Custom Build path possible without smoke and mirrors.



ARTICLE V

Technical Neutrality

The project does not take sides in the architecture wars.

Doobian Core is not a systemd-free distribution. It is also not a systemd distribution. It is both, and neither, depending on the template chosen. The same applies to every other long-running technical disagreement in the Linux world. The project's position on these debates is that the user's position is the only one that matters. A Doobian Standard install with systemd is as legitimate as a Doobian Devuan install with runit. The project's job is to make both work well, not to advocate for one over the other.



ARTICLE VI

Political Neutrality

The project does not recruit a tribe.

The Linux ecosystem has political fault lines that have nothing to do with whether code works. Doobian Core takes no position on those lines. Contributions are evaluated on whether they make the software better for more people, not on who made them or what camps they belong to. The project does not gatekeep by ideology, and does not pander to one. Anyone who can sign up to “Works, and improves. For all people.” is welcome to contribute. Anyone who cannot will find that the project does not engage.



ARTICLE VII

Honest Limits

The project does not promise more than it can deliver.

Modularity has costs. The supported component matrix is finite, not infinite, because every combination on the matrix must actually be tested. The Custom Build path can produce configurations that break, and the project says so plainly at the gate. Some combinations are refused outright — systemd-resolved with a non-systemd init, for example — because they cannot work. The project's neutrality is not a promise of unlimited scope; it is a promise that within the supported scope, the user's choice is honored. A doctrine that only lists virtues is marketing. This is a doctrine, not marketing.



ARTICLE VIII

What This Binds Us To

This doctrine is binding on the project, not aspirational.

The clauses above can be checked. Software that doesn't work violates Article I. A maintainer who narrows the user base violates Article I. A project that loses its modularity violates Article III. A project that picks sides in technical or political fights it could avoid violates Articles V and VI. The doctrine is a contract with users and contributors, and breaking it is grounds for the project to be called out. Doobian Core invites that scrutiny. Without it, a doctrine is just decoration.

Established May 2026. Doctrine v1.0.

Subject to revision only by amendment, never by silent reinterpretation.

BROWN ENTERPRISES

BE AI Research Group